Multicore solutions

CH07-Q01

$$S_8 = 1/(0.1 + 0.9/8) = 4.70588, \qquad E_8 = S_8/8 = 58.8235\%.$$

The limit is 1/0.1=10, approached but not attained for finite workers when parallel time remains positive.

$$S_{128} = 1/(0.03 + 0.97/128) = 26.61123.$$

$$S_{8,5\%} = 1/(0.05 + 0.95/8) = 5.92593, \quad S_{16,10\%} = 1/(0.1 + 0.9/16) = 6.4.$$

The finite comparison favors 6.4 over 5.92593. Infinite limits 20 and 10 cannot replace those finite results.

$$f \geq 100(89)/(90(99)) = 0.998878, \quad 1 - f = 0.001122 = 0.1122\%.$$

CH07-Q02

Heavy work=0.02(10000)=200; the others average 9800/99≈98.99. With indivisible units, 98 receive 99 and one receives 98; the maximum is still 200. Elapsed time is max(200, 98.99)+10=210.

$$S = 10010/210 = 47.67, \qquad E = 47.67/100 = 47.7\%.$$

At 5%, heavy work 500, others 9500/99=95.96:

$$T = \max(500, 9500/99) + 10 = 510, \quad S = 10010/510 = 19.63, \quad E = 19.6\%.$$

For indivisible work in the 5% case, 95 workers receive 96 units and four receive 95; the 500-unit worker remains the critical one.

Work stealing lets idle workers take queued tasks, reducing skew. It adds task/atomic metadata work and migrated cache/network traffic. Excessively fine stealing can lose more than it saves.

Multicore solutions (continued)

CH07-Q03

$$S_4 = 300/80 = 3.75, \qquad E_4 = 3.75/4 = 93.75\%.$$
$$E_{weak} = T_1(W)/T_4(4W) = 75/90 = 5/6 = 83.33\%.$$

With linear single-core cost, T1(4 W)=4(75)=300 s:

$$S_{scaled} = 300/90 = 10/3, \qquad S_{scaled}/4 = 5/6.$$

Weak efficiency compares different total sizes with equal per-core work. Scaled speedup compares the same larger problem on one/four cores. 90/75 is time inflation, not speedup, and weak efficiency must not be divided by four again.

Small baseline=110 t. At 10 workers, T=10 t+100 t/10=20 t, S=5.5; at 100, T=11 t, S=10. Large baseline=10010 t. At 10, T=1010 t, S=9.91089; at 100, T=110 t, S=91.

The small case quickly reaches its fixed 10 t serial floor; the larger parallel work dilutes that cost. The two cases alone are not a weak-scaling experiment: the work increases 100-fold while workers increase 10-fold. Falling efficiency can instead reflect saturated memory, imbalance, synchronization, communication, cache contention, throttling, or noise. A data race is a separate correctness defect.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multicore solutions (continued)

CH07-Q04

U is a UMA shared-memory multiprocessor. M is a private-address-space cluster/message-passing model. Publication needs same-location value correctness and cross-thread ordering. Coherence alone cannot order plain data before ready or remove a race.

Only an acquire actually reading the writer's released 1 synchronizes with it. Writer program order puts data=42 before release; reader program order puts data read after acquire. The transitive happens-before chain gives r=42. Reading 0 does not establish this synchronizes-with relation; entering an unprotected data read cannot claim its guarantee.

Part 4 is a separate all-atomic, all-relaxed program, not the valid release/acquire program in part 3. An allowed observation is

```
writer: data.store(42, relaxed)
writer: ready.store(1, relaxed)
reader: ready.load(relaxed) returns 1
reader: data.load(relaxed) returns initial 0
```

Atomicity is present: neither atomic object tears, and each has its own modification order. What is absent is cross-object synchronization. The relaxed flag does not transmit writer program order into a cross-thread happens-before chain. Restoring the matching release store and acquire load that reads it excludes this outcome.

If conflicting accesses instead use plain non-atomic objects without other synchronization, the C/C++ program has a data race and undefined behavior. A hardware visibility sketch does not restrict that program's language-level outcomes to 0 or 42.

Private spaces cannot exchange portable raw pointers. The sender marshals data, the transport buffers/routes it, and the receiver decodes it. ACK reports delivery under the transport protocol; it does not order unrelated application shared accesses or repair their races.

Multicore solutions (continued)

CH07-Q05

Fine-grained threading rotates each cycle and can hide long/short stalls while interleaving ready threads. Coarse-grained switches on long stalls, interfering less with normal single-thread execution but hiding short dependencies less well. SMT selects ready operations from multiple threads in one issue cycle.

Fine-grained threading need not lower a thread's latency; sharing may increase it while aggregate throughput rises. SMT fills functional-unit holes but shares caches, TLBs, and bandwidth, so working sets may evict one another. If one ready thread already has enough ILP for four slots, SMT's main gain may occur during stalls. Measure fairness, misses, bandwidth, power, interference, and tail latency.

CH07-Q06

Case/group	Count	Scalar cycles/op	Baseline cycles	Eight-lane cycles
A vectorizable	80	1	80	10
A fixed	20	10	200	200
A total	100	unequal	280	210
B vectorizable	80	1	80	10
B fixed	20	1	20	20
B total	100	equal	100	30

$$f_A = 80/280 = 2/7, \qquad S_A = 280/(80/8 + 200) = 4/3.$$

Amdahl gives the same 1/[(1−2/7)+(2/7)/8]. Substituting operation share 0.8 wrongly gives 3.3333, 2.5 times the actual speedup.

$$S_B = 100/(80/8 + 20) = 10/3, \quad S_{B,6} = 100/(\lceil 80/6 \rceil + 20) = 50/17.$$

Fourteen cycles are needed for the 80 operations, so average utilization over eight physical lanes is 80/(14×8)=5/7≈71.43%. Case A instead yields 280/(14+200)=140/107≈1.3084. Vector length specifies valid elements while hardware processes them over its physical lanes; tails use lengths/masks. Fixed-width SIMD usually forms width-sized groups with separate tail handling.

Doubling A's width to 16 gives 280/(5+200)=1.36585, not twice 4/3. Fixed costs remain. Bandwidth, gather/scatter, setup, alignment, dependencies, and lower clocks may reduce real performance further.

Multicore solutions (continued)

CH07-Q07

Mesh horizontal links=4(3)=12 and vertical=12, total 24. Torus adds four horizontal/four vertical wraps, total 32. Mesh degrees are 2 at four corners, 3 at eight edge nodes, and 4 at four interior nodes, degree sum 48 and average 3. Diameter=3+3=6. The cut between columns 1 and 2 has four edges, so bisection is at most 4. For the lower bound, consider the four rows. If all are mixed, each contributes at least one horizontal cut edge. If a full row exists on each side, every column has at least one vertical cut edge. Otherwise a full row occurs on only one side: balance forces exactly one full row, three mixed rows, and at least one vertical cut edge between the full row and an adjacent mixed row. In each case the cut has at least four edges, so the bisection is exactly 4.

Torus degree 4 and each dimension distance$\leq$2 give diameter 4. A balanced column cut crosses four local and four wrap links, proving an upper bound of 8. For the lower bound, the 4$\times$4 torus is isomorphic to Q_4, whose smallest nonzero Laplacian eigenvalue is 2. For any 8/8 cut let x=+1/2 on one side and −1/2 on the other; x is orthogonal to the constant vector and $\| x \|^2$=4. The cut size is $x^T L x \geq 2 \| x \|^2$=8, hence the minimum is exactly 8.

Horizontal wrap pairs are(0, 0)↔(3, 0),(0, 1)↔(3, 1),(0, 2)↔(3, 2),(0, 3)↔(3, 3). Vertical pairs are(0, 0)↔(0, 3),(1, 0)↔(1, 3),(2, 0)↔(2, 3),(3, 0)↔(3, 3). Removing one wrap leaves its endpoints degree 3 rather than 4.

$$|E(K_{16})| = \binom{16}{2} = 120, \quad degree = 15, \quad diameter = 1, \quad bisection = 8(8) = 64.$$

One-way cut bandwidth is 64 b. A central crossbar endpoint usually connects to one switch port, not 15 other endpoints. Internal crosspoints scale as N^2; endpoint port rate and arbitration still limit throughput. Sparse-matrix traffic may have hotspots, locality, directional congestion, or small messages. Routing, flow control, size, and rates matter beyond bisection.

Multicore solutions (continued)

CH07-Q08

Let Q=F/I. Compute time F/PC and memory time F/(IB) imply

$$T \geq \max(F/P_C, F/(IB)), \qquad P = F/T \leq \min(P_C, IB), \quad I_{ridge} = P_C/B = 200/40 = 5.$$

A:0.4(40)=16, so the bandwidth roof is tighter at 16 GFLOP/s. B:12(40)=480, so the compute roof is tighter at 200 GFLOP/s. Actual bottlenecks require measurement.

$$I_D = 10^9/(0.25 \times 10^9) = 4, \quad I_S = 10^9/(4 \times 10^9) = 0.25.$$

$$I_D B_D = 400, \quad I_S B_S = 250 \quad (\text{GFLOP/s}), \qquad T_C = 10^9/(1000 \times 10^9) = 1 \text{ ms}.$$

$$T_D = 2.5 \text{ ms}, \quad T_S = 4 \text{ ms}, \quad P \leq \min(1000, 400, 250) = 250, \quad T \geq 4 \text{ ms}.$$

DRAM ridge 10 puts(4, 400)on its slope. Shared ridge 1 puts(0.25, 250)on its slope; its plateau remains 1000. The combined bound may be shown as(4, 250)using ID, but shared's independent x-coordinate is 0.25.

Measured 30 GFLOP/s is not explained by the bound alone. Diagnose dependencies, instruction mix, actual cache/interface traffic, coalescing, occupancy, imbalance, clock/power, and measurement-phase boundaries. Inspect sectors, cache/TLB counters, stall reasons, and resource utilization before changing the model or optimizing.

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design, Revised Fourth Edition, Chapter 7. Lin Cing, handwritten computer organization notes; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study